

PART #02

# PYTHON PANDAS

@pyadda



## Importing Data

`import pandas as pd`

- `pd.read_csv(filename)` #Coma Seprated file
- `pd.read_table(filename)` #delimited text file (like TSV)
- `pd.read_excel(filename)` #Excel file
- `pd.read_sql(query, connection_object)` #DB
- `pd.read_json(json_string)` #json file

## Exporting Data

- `df.to_csv(filename)`
- `df.to_excel(filename)`
- `df.to_sql(table_name, connection_object)`
- `df.to_json(filename)`

\*df is data frame



@PYADDA

## Viewing/Inspecting Data

1. `df.head(n)` | First n rows of the DataFrame
2. `df.tail(n)` | Last n rows of the DataFrame
3. `df.shape` | Number of rows and columns
4. `df.info()` | Index, Datatype and Memory information
5. `df.describe()` | Summary statistics for numerical columns



@PYADDA

## Importing/reading CSV file

```
1 import pandas as pd
2 df = pd.read_csv("Job.csv")
3 print(df.head())
```

|   | County Name  | 2017     | 2018 | 2019 |
|---|--------------|----------|------|------|
| 0 | South Africa | 0.744814 | 4    | 39   |
| 1 | South Africa | 0.407100 | 46   | 58   |
| 2 | Afganistan   | 0.256150 | 33   | 70   |
| 3 | Afganistan   | 0.805217 | 17   | 53   |
| 4 | Afganistan   | 0.065598 | 24   | 70   |

### **pd.read\_csv(file\_name)**

This method read CSV file and store into pandas df for different operation.

### **df.head(integer)**

This method is used to print top 'n' integer rows of dataframe, by default it fetches top 5 rows.



@PYADDA

## df.tail(n)

This method return last 'n' rows by default 5 rows

```
1 print(df.tail())
```

|    | County Name  | 2017     | 2018 | 2019 |
|----|--------------|----------|------|------|
| 45 | South Africa | 0.613617 | 46   | 71   |
| 46 | South Africa | 0.425667 | 78   | 11   |
| 47 | Indonesia    | 0.289761 | 67   | 22   |
| 48 | South Africa | 0.139750 | 49   | 69   |
| 49 | Sri Lanka    | 0.405664 | 51   | 47   |



@PYADDA

## df.shape

It return tuple of number of rows & number of columns of df

```
1 print(df.shape)
```

```
(50, 4)      (#rows, #columns)
```

```
1 print(df.columns)
```

```
Index(['County Name', '2017', '2018', '2019'], dtype='object')
```

## df.columns

It returns column names

Note : No Brackets after shape & columns



@PYADDA

## df.info()

It returns Index, Datatype and Memory information

```
1 print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 50 entries, 0 to 49  
Data columns (total 4 columns):  
#      Column          Non-Null Count  Dtype  
---  -  
0     County Name      50 non-null    object  
1     2017              50 non-null    float64  
2     2018              50 non-null    int64  
3     2019              50 non-null    int64  
dtypes: float64(1), int64(2), object(1)  
memory usage: 1.4+ KB
```



@PYADDA

## df.describe()

This method return Summary statistics for numerical columns

```
1 print(df.describe())
```

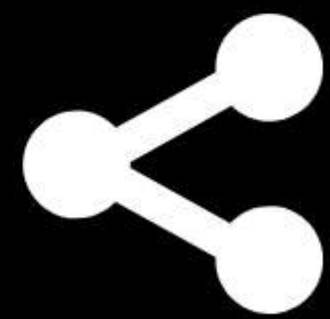
|       | 2017      | 2018      | 2019      |
|-------|-----------|-----------|-----------|
| count | 50.000000 | 50.000000 | 50.000000 |
| mean  | 0.483643  | 42.040000 | 44.880000 |
| std   | 0.291993  | 22.218727 | 21.183012 |
| min   | 0.025243  | 1.000000  | 1.000000  |
| 25%   | 0.261298  | 25.750000 | 28.500000 |
| 50%   | 0.420336  | 46.000000 | 44.000000 |
| 75%   | 0.739338  | 57.250000 | 65.750000 |
| max   | 0.975273  | 78.000000 | 78.000000 |

For categorical column use parameter include='all'

Turn on post Notification ↗



THANK  
YOU



Save for future use



@PYADDA

